

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

TRABAJO AUTÓNOMO INDIVIDUAL SUMATIVO

TEMA:

ANÁLISIS CRÍTICO COMPARATIVO DE UN SISTEMA
DISTRIBUIDO REAL: KUBERNETES (K8S)

MATERIA:

APLICACIONES DISTRIBUIDAS

DOCENTE:

ING. GUERRERO ULLOA GLEISTON CICERON

CURSO:

7MO A

ESTUDIANTE:

JOSÉ ALEJANDRO LOZANO MORALES

FECHA:

21 DE MAYO DE 2026

Resumen

El presente informe técnico expone un análisis crítico e interdisciplinario sobre el comportamiento y las capacidades operativas del orquestador de contenedores Kubernetes como una solución de middleware para aplicaciones distribuidas de alta envergadura. A través de un enfoque arquitectónico sustentado en los compendios de la asignatura, se evalúa de manera rigurosa la gestión de las transparencias de distribución fundamentales tales como acceso, ubicación, replicación y fallas, contrastando sus mecanismos de abstracción frente a las limitaciones físicas de la infraestructura de red. Adicionalmente, el documento examina el motor de persistencia del plano de control bajo las directrices estrictas del Teorema CAP, justificando el comportamiento del sistema mediante el modelado matemático del algoritmo de consenso distribuido Raft. Finalmente, los principios teóricos evaluados son proyectados de forma práctica sobre el Proyecto Fin de Curso (PFC) denominado “TiendaTech”, una plataforma unificada de comercio electrónico para el mercado de hardware con alcance a nivel nacional. La evaluación demuestra cómo la inyección de bucles de reconciliación declarativa y la minimización automatizada del tiempo medio de reparación (MTTR) viabilizan la tolerancia a fallos, la alta disponibilidad y la consistencia transaccional requeridas por una aplicación compleja y concurrente a gran escala.

Palabras Clave: Middleware Distribuido, Transparencias de Distribución, Teorema CAP, Algoritmo Raft, Tolerancia a Fallos, Kubernetes, TiendaTech.

Índice

1. Introducción y Arquitectura General de Kubernetes	3
1.1. Componentes del Control Plane	3
1.2. Componentes de los Nodos Trabajadores (Worker Nodes)	4
2. Análisis de las Transparencias de Distribución	5
2.1. Transparencia de Localización y Acceso	5
2.2. Transparencia de Réplica y Escalado	6
2.3. Transparencia de Fallos	6
3. Evaluación bajo el Teorema CAP	7
3.1. Clasificación de Kubernetes y etcd	7
3.2. El Algoritmo de Consenso Raft	8
4. Modelos de Comunicación en el Clúster	9
4.1. Comunicación Síncrona mediante gRPC y REST	9
4.2. Modelo de Red Plano (IP-per-Pod)	10
5. Mecanismos de Tolerancia a Fallos y Conclusiones	11
5.1. Bucles de Reconciliación y Self-healing	11
5.2. Conclusión Crítica	12
6. Aplicación Práctica y Proyección en el Proyecto Fin de Curso (PFC)	13
7. Conclusiones	14

1. Introducción y Arquitectura General de Kubernetes

Kubernetes (comúnmente abreviado como K8s, haciendo alusión a las ocho letras entre la ‘K’ y la ‘s’) es una plataforma de código abierto de grado orquestador, diseñada originalmente por Google y mantenida actualmente por la Cloud Native Computing Foundation (CNCF). Su propósito primordial es automatizar el despliegue, el escalado dinámico, la interconexión y la administración de aplicaciones basadas en contenedores a lo largo de un grupo de máquinas físicas o virtuales. En el paradigma de la computación distribuida, Kubernetes opera efectivamente como un sistema operativo para clústeres distribuidos, abstrayendo por completo la infraestructura de hardware subyacente y presentando al desarrollador una interfaz unificada y homogénea para la gestión de recursos computacionales [1].

La arquitectura de Kubernetes se fundamenta en un patrón fuertemente desacoplado y modular, estructurado principalmente en dos capas operacionales nítidamente diferenciadas: el nodo maestro o plano de control (*Control Plane*) y los nodos de trabajo o ejecución (*Worker Nodes*). Esta separación limpia de responsabilidades garantiza que las funciones de gobernanza, consistencia global y toma de decisiones estratégicas no interfieran directamente con el rendimiento o el espacio de memoria asignado al procesamiento de las cargas de trabajo de las aplicaciones de los usuarios. Recientemente, la literatura científica ha destacado la alta extensibilidad de este plano de control para coordinar no solo software en la nube, sino también ecosistemas heterogéneos y complejos en la automatización industrial contemporánea [2].

1.1. Componentes del Control Plane

El *Control Plane* actúa como el sistema nervioso central y el núcleo de toma de decisiones del clúster distribuido. Su responsabilidad principal es mantener el “estado deseado” de todo el entorno, monitoreando las variables físicas del sistema y aplicando correcciones automáticas cuando ocurren desviaciones. Este plano de control no ejecuta contenedores de aplicaciones generales, sino que se compone de una suite de servicios altamente especializados:

- **kube-apiserver:** Es el componente medular del Control Plane y actúa como la única puerta de entrada y fachada de comunicación para todo el sistema distribuido. Expone una interfaz de programación de aplicaciones (API) estructurada bajo el estilo arquitectónico REST sobre HTTP/2. Absolutamente todas las interacciones, tanto externas (comandos ejecutados por administradores mediante `kubectl`) como internas (comunicaciones entre agentes de los nodos), deben interceptarse y validarse obligatoriamente a través de este componente. Es el encargado de autenticar las peticiones, autorizar los permisos mediante esquemas RBAC (*Role-Based Access Control*) y validar sintácticamente los objetos antes de consolidarlos en el almacenamiento persistente.
- **etcd:** Es el componente encargado de la persistencia de datos del clúster. Físicamente, consiste en un almacén de clave-valor distribuido, ligero, de código abierto y altamente disponible [3]. Su rol fundamental es salvaguardar la “fuente de la verdad” de Kubernetes; es decir, todos los datos de configuración del clúster, los secretos de la aplicación, los estados de ejecución de los componentes y las declaraciones de recursos existentes se escriben aquí. Al ser un elemento tan crítico, su diseño prioriza de forma estricta

la consistencia matemática absoluta por encima de la disponibilidad inmediata, utilizando algoritmos avanzados de consenso para evitar la corrupción de datos ante fallos parciales de red.

- **kube-scheduler:** Es el motor de asignación de recursos y toma de decisiones logísticas del clúster. Su tarea consiste en monitorizar constantemente la aparición de *Pods* recién creados que aún no tengan asignado un nodo físico para ejecutarse. El planificador evalúa de manera algorítmica las restricciones técnicas del *Pod* (como demandas de CPU, memoria RAM, políticas de afinidad o antiafinidad, y restricciones de etiquetas) y las contrasta con la capacidad disponible real de cada nodo trabajador en ese instante de tiempo, seleccionando el nodo óptimo para albergar la carga.
- **kube-controller-manager:** Este componente compila y ejecuta una serie de subprocesos o bucles de control reguladores empaquetados en un único binario. Cada uno de estos controladores internos opera de forma independiente y asíncrona, encargándose de regular una abstracción específica dentro del clúster (por ejemplo, el *Node Controller* vigila la salud de los servidores, mientras que el *Replication Controller* asegura que el número exacto de réplicas de una aplicación se mantenga idéntico al valor configurado). Su lógica básica responde a un ciclo infinito de retroalimentación: leer el estado real, compararlo con el estado deseado y ejecutar acciones de mitigación si difieren.

1.2. Componentes de los Nodos Trabajadores (Worker Nodes)

Los nodos trabajadores constituyen el músculo ejecutor del sistema distribuido. Son las máquinas físicas o instancias virtuales dedicadas de forma exclusiva a proveer el entorno de procesamiento informático, la memoria volátil y las interfaces de red necesarias para que los contenedores de las aplicaciones del usuario operen de manera eficiente. Cada nodo de trabajo ejecuta un conjunto de agentes esenciales que interactúan armónicamente con el plano de control:

- **kubelet:** Es el agente de software principal que se ejecuta localmente en cada nodo de trabajo del clúster. Actúa como el intermediario directo y el “capataz” subordinado a las órdenes impartidas por el *kube-apiserver*. El *kubelet* no gestiona el clúster global, sino que recibe especificaciones técnicas estructuradas denominadas *PodSpecs* (normalmente formateadas en archivos YAML o JSON) enviadas por el API Server. Su única misión es interactuar con el entorno de ejecución de contenedores local para garantizar que los contenedores descritos en dichas especificaciones se inicien correctamente, se mantengan saludables y tengan asignados sus recursos correspondientes. Si un contenedor muere inesperadamente dentro de su jurisdicción, el *kubelet* detecta el fallo de inmediato y lo reinicia de forma autónoma.
- **kube-proxy:** Es un proxy de red fundamental que se ejecuta de manera local en cada nodo y que implementa la capa de abstracción de red de Kubernetes. Su función principal es gestionar y mantener activas las reglas de red en el host. Estas reglas permiten mapear las direcciones IP virtuales de los objetos de tipo *Service* hacia los *Pods* reales de backend que procesan la información, gestionando el balanceo de carga

de capa 3 y 4 de manera local mediante el uso eficiente de herramientas del kernel del sistema operativo como `iptables` o `IPVS`. Esto garantiza que la comunicación de red interna sea transparente y altamente distribuida.

- **Container Runtime:** Es el entorno o motor de ejecución de bajo nivel encargado de gestionar el ciclo de vida de los contenedores dentro del host físico. Kubernetes no incluye un motor nativo de contenedores, sino que interactúa con tecnologías de terceros mediante una interfaz estándar conocida como CRI (*Container Runtime Interface*). Motores modernos y ligeros como `containerd` o `CRI-O` se encargan de descargar de los repositorios las imágenes de software solicitadas, desempaquetar sus sistemas de archivos aislados, y configurar los mecanismos de aislamiento a nivel de kernel (como *namespaces* y *cgroups*) para que los procesos corran de forma segura.

2. Análisis de las Transparencias de Distribución

El objetivo primordial de un sistema distribuido ideal es lograr que un conjunto de recursos informáticos independientes y geográficamente dispersos se presente ante los usuarios finales y los desarrolladores como si fuese un único sistema centralizado uniforme. Esta propiedad se conoce en la literatura técnica como transparencia de distribución. Kubernetes aborda este desafío de manera excepcional mediante la implementación de capas de abstracción lógica que ocultan por completo la complejidad física, la topología de red cambiante y la volatilidad del hardware subyacente. A continuación, se analiza críticamente cómo K8s gestiona las transparencias clave definidas en el marco de los sistemas distribuidos.

2.1. Transparencia de Localización y Acceso

La transparencia de localización dictamina que los usuarios y las aplicaciones no deben conocer la ubicación física o geográfica de un recurso (como un servidor, un volumen de almacenamiento o una dirección IP específica del host) para poder interactuar con él. Por su parte, la transparencia de acceso exige que las operaciones de comunicación compartan la misma interfaz y semántica, independientemente de si el recurso es local o remoto.

Kubernetes resuelve ambos desafíos de forma unificada mediante la abstracción denominada *Service* (Servicio). En un clúster dinámico, los *Pods* (unidades mínimas de ejecución que albergan los contenedores) son efímeros y volátiles; cada vez que un *Pod* se reinicia, se destruye o se escala, el plugin de red (CNI) le asigna una dirección IP interna completamente nueva. Si los clientes o microservicios dependientes tuvieran que rastrear manualmente estas mutaciones de direccionamiento, el sistema colapsaría debido al acoplamiento.

Un objeto *Service* de Kubernetes actúa como una fachada estática y permanente dotada de un nombre DNS legible (por ejemplo, `mi-servicio.default.svc.cluster.local`) y una dirección IP virtual fija conocida como *ClusterIP*. Cuando un microservicio cliente desea consumir datos de otro componente, efectúa una petición de red estándar hacia ese nombre DNS o IP virtual. El componente local *kube-proxy*, operando en sincronía con el núcleo de red del sistema operativo a través de `iptables` o `IPVS`, intercepta de forma transparente la petición y realiza una traducción de direcciones de red (NAT) para redirigir el tráfico hacia la IP real y efímera de uno de los *Pods* de backend que se encuentran activos en ese preciso

instante. El cliente jamás llega a enterarse en qué nodo del clúster físico corre el contenedor destino ni cuál es su IP real, logrando así una transparencia de localización y acceso perfecta a nivel arquitectónico. Investigaciones recientes demuestran que esta capacidad de abstracción de localización puede extenderse incluso a entornos multi-clúster jerárquicos en el borde (*Edge Computing*), permitiendo invocar recursos remotos mediante nombres locales de manera completamente transparente para el emisor [4].

2.2. Transparencia de Réplica y Escalado

La transparencia de réplica implica que un sistema distribuido puede mantener múltiples copias idénticas de un mismo recurso de datos o componente de software de manera simultánea para mejorar la disponibilidad y el rendimiento, sin que el usuario o las aplicaciones cliente perciban la existencia de dichas duplicaciones. En estrecha relación, la transparencia de escalado permite expandir o contraer la capacidad computacional global del sistema (añadiendo nodos o instancias) de forma dinámica y fluida, sin alterar la estructura interna de la aplicación ni interrumpir los servicios activos.

En el ecosistema de Kubernetes, la transparencia de réplica es gestionada de manera automatizada por el controlador de despliegues (*Deployment Controller*) y el balanceador de carga nativo integrado en los servicios. Cuando un ingeniero define un recurso de tipo *Deployment* en un archivo de configuración e indica, por ejemplo, el parámetro `replicas: 5`, el plano de control no solo distribuye estas cinco instancias idénticas a lo largo de los diferentes nodos trabajadores disponibles para evitar puntos únicos de fallo, sino que el *kube-apiserver* registra dinámicamente las cinco IPs resultantes bajo el mismo objeto *Service*. Cuando una petición externa o interna llega al clúster, el sistema operativo distribuye las cargas equitativamente entre las réplicas utilizando algoritmos de balanceo como *Round-Robin* o conexiones mínimas. El emisor de la trama de red experimenta una sesión de comunicación única, ignorando por completo que su petición fue atendida por la réplica número tres instalada en un servidor físico distinto al de la réplica uno.

El escalado se vuelve transparente mediante componentes como el *Horizontal Pod Autoscaler* (HPA). El HPA monitoriza continuamente métricas analíticas de consumo de recursos extraídas por el *Metrics Server*. Si la demanda de un microservicio supera un umbral crítico de CPU o memoria debido a un pico de tráfico, el controlador incrementa automáticamente el número deseado de réplicas en caliente. Kubernetes descarga la imagen del contenedor, aprovisiona la red y enlaza los nuevos *Pods* al flujo del balanceador de carga de forma transparente, permitiendo absorber incrementos masivos de tráfico global sin necesidad de reconfigurar los componentes clientes ni provocar desconexiones en el sistema distribuido.

2.3. Transparencia de Fallos

La transparencia de fallos constituye uno de los desafíos más complejos e imperativos en la ingeniería de sistemas distribuidos. Esta propiedad establece que el sistema debe poseer la capacidad de enmascarar u ocultar la degradación, mal funcionamiento o colapso completo de componentes individuales (ya sean contenedores de software aberrantes o servidores de hardware enteros), permitiendo que las aplicaciones continúen operando y completando sus tareas pendientes como si el fallo nunca hubiera ocurrido.

Kubernetes materializa esta transparencia mediante un paradigma de diseño centrado en el “bucle de control declarativo de reconciliación constante”. A diferencia de los sistemas imperativos tradicionales donde un administrador debe intervenir para mitigar una crisis, el *Control Plane* de Kubernetes trabaja bajo una filosofía reactiva autorreparable (*self-healing*). A nivel micro, el agente *kubelet* supervisa constantemente los contenedores mediante pruebas de salud especializadas (*Liveness Probes*); si un proceso entra en un estado de congelamiento (*deadlock*) o su consumo de memoria desborda los límites provocando un fallo crítico de ejecución, el *kubelet* destruye el contenedor corrupto y despliega una instancia limpia en milisegundos de forma local.

A nivel macro, el *Node Lifecycle Controller* monitoriza los latidos de corazón (*heartbeats*) que cada nodo trabajador envía periódicamente al plano de control. Si un servidor físico sufre un corte de energía, una avería en sus tarjetas de red o un colapso de hardware masivo, dejará de reportar su estado. Tras expirar un periodo de gracia parametrizado (típicamente cinco minutos), el plano de control marca formalmente al nodo afectado como `NotReady` e inicia un proceso automático de evacuación y reprogramación forzada. El *kube-scheduler* identifica inmediatamente todos los *Pods* que se encontraban atrapados en la máquina siniestrada y los reasigna de manera transparente hacia otros nodos sanos del clúster que cuenten con capacidad ociosa. Al mismo tiempo, las reglas de red administradas por *kube-proxy* se actualizan en tiempo real a nivel global para dejar de enviar tráfico al nodo caído. Desde la perspectiva del usuario final que consume la aplicación web a través de internet, el servicio nunca experimenta una interrupción catastrófica, enmascarando el fallo físico del hardware detrás de la redundancia y la automatización del clúster.

3. Evaluación bajo el Teorema CAP

El Teorema CAP, postulado inicialmente por el científico de la computación Eric Brewer en el año 2000, constituye uno de los pilares teóricos fundamentales para el diseño y análisis de cualquier sistema distribuido. Este teorema establece formalmente que es matemáticamente imposible que un sistema de datos distribuido garantice simultáneamente tres propiedades críticas de manera absoluta en presencia de un particionamiento de red: Consistencia (*Consistency*), Disponibilidad (*Availability*) y Tolerancia al Particionado (*Partition Tolerance*).

Dado que las redes físicas reales son inherentemente falibles y sufrirán retrasos o pérdidas de paquetes inevitables (lo que obliga a asumir la Tolerancia al Particionado, o propiedad P, como una realidad técnica invariable), cualquier sistema distribuido real se ve forzado a realizar una elección de diseño crítica cuando ocurre una partición: o bien prioriza la Consistencia absoluta sacrificando la Disponibilidad inmediata (sistemas CP), o bien mantiene la Disponibilidad sacrificando la consistencia uniforme a través de los nodos (sistemas AP).

3.1. Clasificación de Kubernetes y etcd

Para evaluar a Kubernetes bajo el prisma del Teorema CAP, es indispensable centrar la atención en su componente más crítico de persistencia de estado: el almacén de clave-valor *etcd*. Dado que Kubernetes es un sistema conducido por estados declarativos, la integridad de todo el clúster depende de que la “fuente de la verdad” almacenada sea idéntica y libre

de contradicciones para todos los componentes analíticos que la consulten. Por esta razón, el diseño arquitectónico de *etcd* (y por extensión, el núcleo de sincronización de Kubernetes) se clasifica estrictamente como un sistema de tipo **CP**.

La elección de priorizar la consistencia fuerte (*Strong Consistency*) implica que, ante un escenario de partición de red que divida el clúster en dos segmentos aislados, Kubernetes prefiere rechazar operaciones de escritura o responder con un error de infraestructura (pérdida de disponibilidad temporal) antes que permitir un escenario catastrófico de *Split-Brain* (cerebro dividido), donde ambas facciones registren datos divergentes de forma descoordinada.

La literatura científica contemporánea recalca que este balance crítico entre la consistencia estricta de datos y su disponibilidad inmediata en tiempo real es una de las tensiones arquitectónicas de red más complejas de mitigar en entornos altamente distribuidos

[5]. Por ejemplo, si una partición de red aislara de forma estricta los nodos maestros y no se pudiera determinar con certeza matemática el estado real de un recurso, el sistema bloqueará temporalmente las mutaciones sobre ese recurso. Si un administrador intentara escalar un despliegue mediante el comando `kubect1 scale`, el *kube-apiserver* rechazará la petición o será incapaz de consolidarla en *etcd*, garantizando que el sistema no sufra corrupciones lógicas a expensas de sacrificar la tolerancia al retardo del usuario.

3.2. El Algoritmo de Consenso Raft

Para hacer viable un modelo CP de alta disponibilidad relativa bajo condiciones normales, *etcd* delega la gobernanza de sus datos al algoritmo de consenso distribuido **Raft**. La misión de Raft es asegurar que una máquina de estados replicada opere de manera idéntica en múltiples servidores independientes, garantizando que una escritura sea confirmada únicamente cuando se ha vuelto completamente irreversible a lo largo de la red.

El funcionamiento matemático y operativo de Raft en Kubernetes se fundamenta en el concepto estricto de **quórum**. Para que un clúster de *etcd* compuesto por un número total de nodos N pueda procesar y asentar una transacción o elegir un nuevo nodo líder, se requiere obligatoriamente el consentimiento y confirmación de la mayoría absoluta de sus miembros. Esta relación matemática se expresa formalmente mediante la siguiente ecuación:

$$Q = \left\lfloor \frac{N}{2} \right\rfloor + 1 \quad (1)$$

Donde Q representa el número mínimo de nodos operativos necesarios para alcanzar quórum. A partir de esta propiedad, la tolerancia máxima a fallos simultáneos de nodos (F) que el clúster puede soportar sin perder su capacidad de cómputo y consistencia se calcula de la siguiente manera:

$$F = \left\lfloor \frac{N - 1}{2} \right\rfloor \quad (2)$$

Esta fundamentación matemática justifica por qué los clústeres de producción de *etcd* en Kubernetes se despliegan invariablemente utilizando un número impar de nodos (típicamente 3 o 5 instancias). Consideremos dos escenarios analíticos comparativos para ilustrar la resiliencia del sistema bajo el Teorema CAP:

1. **Clúster de 3 nodos ($N = 3$):** Aplicando la ecuación (1), el quórum requerido es $Q = \lfloor 3/2 \rfloor + 1 = 2$ nodos. De acuerdo con la ecuación (2), el sistema tolera el fallo de $F = \lfloor (3 - 1)/2 \rfloor = 1$ nodo. Si una partición de red aísla a un solo nodo, los 2 nodos restantes mantienen la comunicación, alcanzan el quórum mínimo requerido ($2 \geq 2$) y el clúster de Kubernetes continúa procesando lecturas y escrituras consistentemente.
2. **Clúster de 4 nodos ($N = 4$):** El quórum requerido se eleva a $Q = \lfloor 4/2 \rfloor + 1 = 3$ nodos. La tolerancia a fallos sigue siendo de solo $F = \lfloor (4 - 1)/2 \rfloor = 1$ nodo. Si una partición de red simétrica divide el clúster exactamente a la mitad (2 nodos a cada lado), ninguna de las dos facciones aisladas podrá alcanzar los 3 nodos exigidos por el quórum ($2 < 3$). En consecuencia, todo el plano de control de *etcd* se bloquea inmediatamente y entra en un estado defensivo de solo lectura, rechazando cualquier cambio de configuración. Esto demuestra empíricamente que añadir un cuarto nodo no incrementa la resiliencia ante fallos, sino que eleva la probabilidad estadística de perder la disponibilidad general.

Cuando un nodo líder de Raft recibe una propuesta de modificación de estado desde el *kube-apiserver*, la escribe localmente en su registro log e intenta replicarla de forma síncrona a todos los nodos seguidores. Solo cuando el líder recibe confirmaciones de un número de nodos que iguale o supere el valor de Q , la transacción se considera formalmente consolidada (*committed*). Inmediatamente después, se retorna una respuesta exitosa al cliente. Si la partición de red impide alcanzar ese umbral mayoritario, el cambio jamás se consolida, preservando la consistencia lineal del sistema distribuido a expensas de la disponibilidad de la API.

No obstante, investigaciones críticas han demostrado que si bien Raft es robusto ante fallos por parada simples (*fail-stop*), el plano de control requiere un análisis extendido cuando surgen comportamientos arbitrarios o corruptos a nivel de red, conocidos comúnmente como fallos bizantinos [6].

4. Modelos de Comunicación en el Clúster

La topología de red y los patrones de interacción dentro de un clúster de Kubernetes constituyen un caso de estudio excepcional en la ingeniería de sistemas distribuidos. Al ser una plataforma diseñada bajo un paradigma de microservicios altamente desacoplados, K8s rechaza el uso de un modelo de comunicación único y monolítico. En su lugar, implementa un enfoque híbrido coordinado donde coexisten protocolos síncronos fuertemente tipados para tareas de gobernanza y control, flujos asíncronos orientados a eventos para la sincronización del estado lúdico, y un modelo de enrutamiento plano de capa 3 para el intercambio masivo de datos entre aplicaciones.

4.1. Comunicación Síncrona mediante gRPC y REST

El flujo de control y coordinación estratégica de Kubernetes es predominantemente síncrono y estructurado en torno al componente central *kube-apiserver*. Cuando los agentes internos (como el *kubelet* o el *kube-scheduler*) o los clientes externos interactúan con el clúster, las transacciones operan bajo llamadas formales basadas en interfaces binarias optimizadas.

Históricamente, el plano de control dependía de interfaces REST tradicionales con serialización JSON sobre conexiones HTTP estándar. No obstante, para optimizar la latencia, el rendimiento y reducir la sobrecarga computacional (*overhead*) en entornos de gran escala, la comunicación interna ha migrado masivamente hacia el framework **gRPC** (*Google Remote Procedure Call*), el cual opera de manera nativa sobre el protocolo de transporte **HTTP/2**. gRPC utiliza *Protocol Buffers* como lenguaje de descripción de interfaz (IDL) y formato de empaquetamiento de mensajes binarios. Esto permite que el intercambio de variables lógicas entre el *kube-apiserver* y los componentes subordinados sea extremadamente compacto, reduciendo drásticamente la latencia de serialización y permitiendo la reutilización de conexiones mediante la multiplexación de HTTP/2. En arquitecturas masivas de producción contemporáneas, la optimización de estos hilos de interconexión se vuelve un requisito crítico no negociable para mitigar los cuellos de botella de red [7].

A pesar de la naturaleza síncrona de estas llamadas remotas, Kubernetes mitiga el bloqueo tradicional de los hilos de ejecución mediante la implementación de la **Watch API**. El *kube-apiserver* expone *endpoints* HTTP optimizados que permiten a los controladores abrir conexiones persistentes de larga duración. En lugar de realizar consultas repetitivas e ineficientes (*polling*) para verificar si el estado ha cambiado, los controladores se suscriben a un flujo asíncrono de eventos (*Stream*). Cuando ocurre una mutación en *etcd*, el API Server la transmite inmediatamente a través de la conexión activa a los agentes interesados. Este modelo reactivo híbrido garantiza que las decisiones lógicas se ejecuten en tiempo cuasi-real, optimizando el ancho de banda del clúster distribuido.

4.2. Modelo de Red Plano (IP-per-Pod)

A nivel de transporte de datos —es decir, la comunicación directa entre los contenedores de las aplicaciones de los usuarios— Kubernetes impone un axioma arquitectónico fundamental conocido como el modelo de red **IP-per-Pod** (IP por cada Pod). Este principio técnico estipula tres reglas de enrutamiento invariables de capa 3 que deben cumplirse estrictamente dentro del sistema distribuido:

1. Todos los *Pods* pueden comunicarse con cualquier otro *Pod* en el clúster sin la mediación de mecanismos de traducción de direcciones de red (NAT).
2. Todos los agentes de hardware en un nodo (por ejemplo, el *kubelet*) pueden comunicarse con todos los *Pods* alojados en ese mismo nodo o en nodos remotos de forma directa.
3. La dirección IP que un *Pod* ve como su propia identidad interna es exactamente la misma dirección IP que los demás componentes del clúster registran para comunicarse con él.

Para materializar este modelo plano y homogéneo sobre una infraestructura física fragmentada, Kubernetes delega la configuración del entorno a complementos de terceros a través de la interfaz estándar **CNI** (*Container Network Interface*), utilizando tecnologías como Calico, Flannel o Cilium.

Desde la perspectiva teórica de redes distribuidas, la CNI aprovisiona una red superpuesta o de superposición (*Overlay Network*) [5]. Cuando un *Pod* en el Nodo A desea enviar un paquete de datos a un *Pod* ubicado físicamente en el Nodo B, el paquete se encapsula localmente

en el host de origen utilizando protocolos de tunelización industrial como **VXLAN** (*Virtual Extensible LAN*) o **Geneve**. El paquete encapsulado viaja a través de los conmutadores y enrutadores físicos de la red del centro de datos como tráfico estándar entre servidores. Al llegar al nodo de destino, el motor de la CNI intercepta la trama, remueve la cabecera de encapsulación y entrega el paquete original intacto al contenedor de destino. Aunque este enfoque es el estándar, la literatura científica de vanguardia estudia actualmente el uso de tecnologías avanzadas como eBPF y XDP a nivel de kernel para saltarse el procesamiento pesado de **iptables** y acelerar drásticamente el flujo de paquetes [4].

Este enfoque elimina por completo la complejidad de gestión de puertos que plagaba a los primeros sistemas de orquestación basados en Docker (donde se requería mapear puertos dinámicos del host al contenedor). Al tratar a cada *Pod* como un host de red real dotado de una IP enrutable globalmente dentro del clúster, Kubernetes simula un modelo cliente-servidor clásico altamente limpio. Esto facilita enormemente el despliegue de arquitecturas de microservicios, ya que los desarrolladores pueden asumir la existencia de conectividad directa completa, aislando las políticas de seguridad mediante abstracciones lógicas de software independientes conocidas como *Network Policies*.

5. Mecanismos de Tolerancia a Fallos y Conclusiones

La tolerancia a fallos en un sistema distribuido de la magnitud de Kubernetes no se concibe como una característica aislada o añadida de forma superficial, sino como una propiedad emergente de su propio diseño arquitectónico. Al asumir como premisa fundamental que cualquier infraestructura física o virtual de red fallará eventualmente, K8s estructuró su núcleo operativo alrededor de un modelo asíncrono y desacoplado de agentes inteligentes encargados de la vigilancia y reparación automatizada de los componentes del clúster.

5.1. Bucles de Reconciliación y Self-healing

El mecanismo medular que hace posible la autorreparación (*self-healing*) en Kubernetes es el denominado **bucle de reconciliación**. Desde una perspectiva algorítmica, este bucle constituye un ciclo infinito de control de retroalimentación ejecutado por el *kube-controller-manager*. El algoritmo fundamental de este proceso sigue tres pasos lógicos iterativos invariables:

[H] Algoritmo Genérico de Reconciliación de Kubernetes [1] true *EstadoDeseado* $\leftarrow$ Leer el estado declarativo desde *etcd* *EstadoReal* $\leftarrow$ Monitorear la infraestructura real mediante agentes *EstadoReal* $\neq$ *EstadoDeseado* Calcular la brecha o diferencia lógica de recursos Ejecutar acciones de mitigación imperativas para converger Esperar un intervalo de tiempo regulado Δt

Para que este bucle funcione a nivel de microservicio individual, Kubernetes delega en el agente *kubelet* la monitorización de la salud de las aplicaciones mediante tres tipos de pruebas lógicas configurables conocidas como **Probes**:

- **Liveness Probe (Prueba de supervivencia)**: Determina si un contenedor de aplicación sigue vivo o ha entrado en un estado de congelamiento permanente (*deadlock*).

Si la prueba falla, el *kubelet* destruye inmediatamente el contenedor y crea una réplica totalmente limpia de forma automática.

- **Readiness Probe (Prueba de preparación):** Evalúa si un contenedor está listo para recibir tráfico de red de usuarios reales. Si una aplicación está arrancando o ejecutando migraciones internas, la prueba retorna un valor negativo, provocando que el balanceador de carga del *Service* remueva temporalmente la IP del *Pod* de sus tablas de enrutamiento.
- **Startup Probe (Prueba de inicio):** Desactiva las otras dos pruebas durante el arranque inicial de aplicaciones de carga lenta, garantizando que el sistema no destruya por error un proceso que simplemente está tardando más de lo habitual en iniciar.

A gran escala, si un nodo de trabajo completo colapsa físicamente, las reglas del *Node Lifecycle Controller* intervienen de manera drástica. Al no recibir señales de vida o *heartbeats* a través de la API, el plano de control marca de forma asíncrona todos los recursos del nodo caído como perdidos y ordena al planificador (*scheduler*) recrear instantáneamente los *Pods* evacuados en máquinas sanas. En infraestructuras hiper-masivas de producción, el diagnóstico rápido y la interceptación automatizada de fallos por caída de hardware (*fail-stop*) resultan vitales para sostener la continuidad operativa de los servicios [7].

Para validar empíricamente que estos bucles de autorreparación actúen con presteza, las tendencias modernas de ingeniería de software aplican la **Ingeniería del Caos** (*Chaos Engineering*), la cual consiste en destruir intencionalmente recursos o inyectar latencia en caliente para evaluar proactivamente los límites de resiliencia del clúster ante fallos no planificados [8].

5.2. Conclusión Crítica

El análisis crítico comparativo de Kubernetes revela que la orquestación moderna de sistemas distribuidos ha alcanzado un nivel de abstracción sin precedentes, pero a costa de introducir una severa complejidad operativa y arquitectónica. Kubernetes demuestra empíricamente que es posible enmascarar las deficiencias y la volatilidad del hardware físico mediante la automatización de software y la imposición de modelos estrictos de diseño declarativo.

Bunder la óptica del Teorema CAP, la decisión de erigir la persistencia del sistema sobre una base de datos de tipo CP como *etcd* es matemáticamente acertada. En un entorno encargado de coordinar redes superpuestas, almacenamiento virtual y ciclos de vida de miles de procesos, permitir inconsistencias en el estado global para favorecer la disponibilidad inmediata provocaría colisiones de red catastróficas y fugas de recursos indetectables. El algoritmo Raft proporciona el ancla de consistencia linealizable indispensable para que las abstracciones lógicas de las transparencias de distribución actúen de manera predecible.

Sin embargo, desde el punto de vista de la ingeniería de software pragmática, Kubernetes no constituye una solución universal. El “impuesto de abstracción” que paga un clúster es sumamente elevado: el tráfico de red sufre penalizaciones de latencia debido a la encapsulación de la CNI, y el plano de control consume recursos de procesamiento considerables de manera constante solo para mantener activos los bucles de reconciliación y la serialización binaria de gRPC. Por lo tanto, para sistemas distribuidos monolíticos de baja escala, la introducción

de Kubernetes puede inducir un exceso de ingeniería (*over-engineering*). No obstante, para ecosistemas globales basados en microservicios, donde la elasticidad horizontal, la alta disponibilidad y la inmunidad automatizada ante fallos de hardware sean requisitos de negocio no negociables, Kubernetes se consolida como la arquitectura de referencia estándar más sólida de la computación contemporánea.

6. Aplicación Práctica y Proyección en el Proyecto Fin de Curso (PFC)

Para consolidar el análisis crítico de Kubernetes dentro de un escenario práctico y alineado con los objetivos de nuestra carrera, se evalúa su proyección sobre el Proyecto Fin de Curso (PFC), el cual consiste en **TiendaTech: una plataforma unificada de comercio electrónico para la venta de componentes de hardware y armado de computadoras personalizadas con alcance a nivel nacional**. A diferencia de un mercado virtual multifirma, TiendaTech opera como una entidad centralizada que asume la responsabilidad absoluta del inventario, la facturación y el despacho de mercancía hacia todo el país, integrando además una herramienta interactiva que valida algorítmicamente la compatibilidad técnica entre las piezas seleccionadas por el usuario.

La adopción de Kubernetes como middleware distribuido resulta un factor estratégico e indispensable para viabilizar la operatividad, el rendimiento y la resiliencia de la infraestructura de TiendaTech ante la demanda del mercado nacional:

1. **Aislamiento y Escalado del Motor de Compatibilidad (Arma tu PC):** El configurador de computadoras no es un componente estático; requiere procesar en tiempo real complejas reglas lógicas (verificar sockets de placas madres, dimensiones de chasis, consumos energéticos en vatios de fuentes de poder y tipos de memoria DDR). Si cientos de usuarios en diferentes provincias utilizan la herramienta de diseño de forma simultánea durante un lanzamiento de hardware, el consumo de CPU y memoria se dispararía drásticamente. Bajo Kubernetes, este módulo lógico puede aislarse en microservicios independientes que escalan de manera elástica mediante el *Horizontal Pod Autoscaler* (HPA), evitando que el procesamiento pesado de compatibilidades ralentice el catálogo general o colapse la pasarela de pagos.
2. **Consistencia Fuerte en el Inventario Nacional Centralizado:** Al ser TiendaTech el único proveedor que despacha a todo el país, la gestión de existencias exige una sincronización transaccional en tiempo real y sin fisuras. Si se dispone de la última unidad de una tarjeta gráfica de alta gama, el sistema debe garantizar que en el instante exacto en que un cliente en Quito confirma su compra, el inventario se actualice de forma linealizable (Consistencia en sistemas CP), reflejando inmediatamente el estado como “Agotado” para cualquier otro comprador en Guayaquil o Cuenca. Los bucles de reconciliación de Kubernetes aseguran que los microservicios de inventario operen sobre un estado siempre consistente, impidiendo la venta de hardware inexistente.
3. **Resiliencia (*Self-Healing*) Frente a Eventos de Alta Demanda:** El mercado tecnológico se caracteriza por picos agresivos de tráfico web debido a promociones

nacionales o reabastecimiento de componentes críticos (*restocks*). La caída del servidor centralizado paralizaría por completo los ingresos del negocio a nivel nacional. Gracias a las sondas lógicas de monitorización local (*Liveness* y *Readiness Probes*) ejecutadas por el agente *kubelet*, cualquier fallo imprevisto o congelamiento de hilos en el servicio de facturación o envíos es detectado y reparado automáticamente en milisegundos a través de la recreación de contenedores sanos, reduciendo el Tiempo Medio de Reparación (MTTR) y blindando la disponibilidad del e-commerce.

En conclusión, la proyección técnica demuestra que para las demandas de carga asimétrica y la complejidad lógica que maneja **TiendaTech** a escala nacional, Kubernetes no representa un exceso de ingeniería (*over-engineering*). Por el contrario, se consolida como el cimiento tecnológico obligatorio para transformar una aplicación web convencional en un sistema distribuido robusto, altamente disponible y con la capacidad de cómputo necesaria para soportar el mercado de hardware de todo el país.

7. Conclusiones

Tras realizar el análisis crítico y exhaustivo de Kubernetes como un sistema distribuido real bajo los fundamentos teóricos de la Unidad 1, se consolidan las siguientes conclusiones de ingeniería:

- **Efectividad en la Abstracción de la Distribución:** Kubernetes demuestra que las transparencias de localización, acceso, réplica y fallos pueden ser implementadas con éxito a nivel de software de infraestructura. El uso de abstracciones lógicas como los *Services* y el enrutamiento gestionado por *kube-proxy* logra enmascarar por completo la volatilidad física de las IPs de los contenedores, permitiendo que un entorno fragmentado sea percibido por los desarrolladores y usuarios como un sistema unificado y cohesivo.
- **Coherencia con los Límites Matemáticos del Teorema CAP:** El diseño de Kubernetes respeta rigurosamente el teorema de Eric Brewer al delegar la gestión del estado del clúster a un almacén CP como *etcd*. El uso de algoritmos de consenso basados en quórum como Raft prueba que, en entornos distribuidos críticos, es preferible sacrificar temporalmente la disponibilidad operativa para mutaciones del sistema antes que arriesgarse a una corrupción o inconsistencia de datos provocada por un particionamiento de red.
- **Eficiencia Mediante Modelos de Comunicación Híbridos:** La coexistencia de llamadas síncronas serializadas binariamente mediante gRPC para el control, flujos reactivos mediante la *Watch API* y una red superpuesta plana *IP-per-Pod* para los datos, evidencia un equilibrio óptimo entre control y rendimiento. Esto elimina la necesidad de NAT complejas en las capas de aplicación, emulando la simplicidad del modelo cliente-servidor tradicional a escala masiva.
- **Resiliencia Basada en el Paradigma Declarativo:** La tolerancia a fallos en Kubernetes es altamente efectiva debido a la sustitución de la intervención humana imperativa

por bucles de reconciliación algorítmica infinita. Al evaluar constantemente la brecha entre el estado deseado y el real, combinado con monitoreos lógicos locales mediante *Probes*, K8s redefine la resiliencia en sistemas modernos, permitiendo que el software absorba fallos masivos de hardware sin impactar la continuidad del servicio del usuario final.

Referencias

- [1] B. Burns, B. Grant, D. Oppenheimer, E. Brewer y J. Wilkes, “Borg, Omega, and Kubernetes,” *ACM Queue*, vol. 14, n.º 1, págs. 70-93, 2016.
- [2] N. Giulianelli, A. Sabbioni y A. Bujari, “KOTO: A Kubernetes-Based Platform for Secure and Scalable OT Orchestration in Industry 5.0,” *Proceedings of ACM/IEEE Conferences*, págs. 1-8, 2025.
- [3] CoreOS. “etcd: Un almacén de clave-valor distribuido y confiable para los datos más críticos de un sistema distribuido,” Documentación oficial de arquitectura, visitado 20 de mayo de 2026. dirección: <https://etcd.io/>.
- [4] Y. Cui, W. Lu, L. Shen y H. Zhang, “EdgeConnector: Enabling Seamless and Efficient Cross-Cluster Device Access in Edge Environment,” en *Proceedings of the 26th ACM Middleware Conference*, 2025, págs. 1-12.
- [5] A. Calagna, S. Ravera y C. F. Chiasserini, “Enabling efficient collection and usage of network performance metrics at the edge,” *Computer Networks*, vol. 262, pág. 111158, 2025.
- [6] G. M. Diouf, H. Elbiaze y W. Jaafar, “On Byzantine fault tolerance in multi-master Kubernetes clusters,” *Future Generation Computer Systems*, vol. 109, págs. 407-419, 2020.
- [7] B. Wan et al., “Robust LLM Training Infrastructure at ByteDance,” en *Proceedings of International Conference on Software Engineering (ICSE)*, Hong Kong, 2025, págs. 1-14.
- [8] E. U. Aktas, B. Tuzlutas y B. Yesiltas, “Designing a Custom Chaos Engineering Framework for Enhanced System Resilience at Softtech,” *ACM Reference Format*, págs. 1-10, 2025.